Surya Siddhanta Computational Library with AI-Powered Query Interface

A Project Report submitted by

Ashutosh Nigam

M25AI2006

in partial fulfillment of the requirements for the award of the degree of

M.Tech.



Indian Institute of Technology Jodhpur

Department of Computer Science and Engineering

November 2026

Declaration

I hereby declare that the work presented in this Project Report titled **Surya Siddhanta Computational Library with AI-Powered Query Interface** submitted to the Indian Institute of Technology Jodhpur in partial fulfillment of the requirements for the award of the degree of **M.Tech.**, is a Bonafide record of the research work carried out under the supervision of **Dr. Dinesh Mohan Joshi**. The contents of this Project Report in full or in parts, have not been submitted to, and will not be submitted by me to, any other Institute or University in India or abroad for the award of any degree or diploma.

Signature

Ashutosh Nigam

M25AI2006

Certificate

This is to certify that the Project Report titled **Surya Siddhanta Computational Library with AI-Powered Query Interface**, submitted by **Ashutosh Nigam (M25AI2006)** to the I**ndian Institute of Technology Jodhpur** for the award of the degree of **M.Tech.**, is a bonafide record of the research work done by him/her under my supervision. To the best of my knowledge, the contents of this report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree or diploma.

Signature

Dr. Dinesh Mohan Joshi

Abstract

This project implements the computable methods of the Surya Siddhanta in a reusable Python library and provides web and AI interfaces for using them. One module corresponds to each of the fourteen chapters. The library covers planetary motion and corrections, eclipses, place and time, conjunctions, stars, visibility, lunar phases, cosmography, instruments, and calendar reckoning. The chapter functions perform the astronomical calculations; the interfaces present their results and source references.

The implementation follows Burgess's English translation and checks selected calculations against worked examples in Mahavir Prasad Srivastava's Hindi Vigyan Bhashya. Verse-based tests and examples exposed errors in inner-planet motion, the manda anomaly, and the Ujjain time base. Separate work extracted Hindi and English source material, explored several OCR models, and produced a searchable corpus. The app bundles 500 numbered English verses across all fourteen chapters; the Hindi OCR outputs remain separate. React and FastAPI support chapter exploration, eight learning paths, and a Research Explorer, while OpenRouter or DeepSeek provides the conversational model. A comparison with JPL DE440 evaluates the adopted longitude calculations under stated frame and time conventions. These checks establish specific software behavior and numerical differences; they do not establish educational effectiveness or reproduce every aspect of historical practice.

Contents

	page
Abstract	i
Table of Contents	ii
1. Introduction	1
2. Literature Survey	2
3. Problem Definition and Objectives	3
4. Methodology	4
5. Results and Findings	25
6. Summary and Future Plan of Work	28
References	30

1. Introduction

The *Sūrya Siddhānta* gives a connected set of rules for astronomical calculations. Its fourteen chapters begin with measures of time and mean planetary motions, then proceed through true positions, local geometry, eclipses, conjunctions, stars, visibility, the lunar crescent, instruments, and calendar reckoning. A verbal account can leave the meaning of a quantity or the clock used in a calculation unresolved. A working program requires an explicit choice for each ambiguity, and those choices appear in its output.

The software also records how the text has been interpreted. A longitude result needs an epoch and a coordinate convention. An eclipse calculation also needs an observer. An explanation drawn from the corpus needs a chapter and verse reference. Recording these details lets a reader follow the calculation and question the choices behind it.

The project serves programmers who need reusable functions and readers who want to inspect the astronomical rules. Both need to know whether a result follows a verse, a translator's commentary, or an implementation convention. Chapter-based modules, verse references, and intermediate values make those distinctions visible. The reading and calculation interfaces use the same source references.

1.1 Abbreviations and terms

The abbreviation SS refers to the Sūrya Siddhānta. JD is Julian Day, a continuous numerical date representation. Ahargaṇa is the elapsed day count used by the text. Jyā is the tabulated sine. Manda and śīghra name the two principal corrections to mean planetary position. DE440 is the Jet Propulsion Laboratory's modern planetary and lunar ephemeris used as a reference in a separate study. MCP is the Model Context Protocol through which an external model client can call the project's tools.

2. Literature Survey

2.1 Source texts and editorial approach

The principal source of rules and constants is Ebenezer Burgess's 1860 English translation of the *Sūrya Siddhānta*. The project also uses worked examples in Mahavir Prasad Srivastava's Hindi *Sūrya Siddhānta: Vigyan Bhashya*. Those examples give intermediate values for specific dates and places, which are useful for checking signs, units, and the order of operations. The Hindi material has been consulted through OCR, so a doubtful digit must be checked against the page image. OCR can confuse the period typeface's nine and six; a disagreement is not automatically a software defect.

The code identifies where a procedure follows the verse and where commentary supplies an otherwise missing step. This distinction matters for the instruments in Chapter 13 and other passages whose descriptions do not give a complete numerical algorithm. The project implements a defensible reading of the computable core, not a critical edition of the Sanskrit text, and not every historical practitioner's method.

Burgess's translation is not treated as a ready-made computer program. A verse may prescribe a relation but leave the starting value, order of correction, or instrument construction implicit. Burgess's notes sometimes supply a method; where the project follows that reconstruction, it records the source. The second text supplies worked arithmetic, but does not justify removing a difference between the verse and commentary. This separation helps explain a mismatch without changing a historically motivated function merely to agree with one printed table. It also shows later readers where an interpretation can be revised.

Source attribution is especially important where a printed translation, its notes, and a later commentary offer different amounts of detail. The project's implementation manuscript classifies disagreements with worked examples as a software defect, an intentional deviation, a source problem, or a rule not yet implemented. This scheme prevents the historical rule from being changed simply to make every test match a printed number. It also prevents the opposite mistake of dismissing every mismatch as a misprint. Each classification needs evidence: a page image for a suspected OCR error, a verse reference for a rule, and an intermediate trace for a calculation.

3. Problem Definition And Objectives

3.1 Project objectives

The project implements the text's computable methods in Python and checks them against source verses and numerical examples. The library is the calculation core. Web pages and the conversational assistant let users request results, inspect verses and explore diagrams without duplicating the chapter arithmetic. Intermediate values and source references make discrepancies easier to investigate.

The completed work includes the calculation library, source-text extraction and corpus preparation, browser interfaces, an AI query interface, and numerical evaluation. The main objectives are to make the historical procedures reusable, show how each calculation follows its source, and compare the resulting longitudes with a clearly defined modern reference.

3.2 Project scope and completed work

The calculation library covers the computable core of all fourteen chapters. The repository includes a chapter module for each chapter, a per-planet interface, panchanga functions, REST routes, browser interfaces, a tool-using chatbot, and an MCP server. Verse-anchored tests, worked-example checks, and a DE440 longitude comparison address distinct questions: whether a rule was transcribed consistently, whether it reproduces a printed example, and how the implementation compares with modern positions. These checks answer different questions and should not be combined into a single claim of accuracy.

The completed computational work follows the chapter structure. Chapters 1–3 cover mean motions, true positions, direction, place, and time. Chapters 4–6 cover eclipses and their projected geometry. Chapters 7–11 address conjunctions, stars, heliacal phenomena, the Moon, and pātas. Chapters 12–14 cover cosmography, instruments, and time reckoning. The applications let a user browse verses, run a calculation through a web route, or call a tool through the chatbot or MCP server. The adopted constants are explicit and are not silently combined with a modern ephemeris.

The deliverables include an installable Python library with chapter-based and planet-based imports, eight guided learning paths, a Universe Explorer, and a Research Explorer. The corpus work includes Hindi OCR and translation outputs and a corrected Burgess export containing 500 numbered English verses across fourteen chapters. The chatbot calls the shared calculation and retrieval tools. Tests and

two project manuscripts document the implementation and its numerical evaluation. The report identifies the remaining algorithmic gaps and transcription issues.

4. Methodology

4.1 Time and coordinate conventions

Julian Day is the shared numerical date representation throughout the library. The historical day count is anchored to the Kali epoch at the meridian through Ujjain. The project distinguishes dates interpreted in Ujjain local mean time from dates interpreted in Universal Time. Passing the former to a sidereal-time routine as if it were the latter rotates local sky geometry by about 76 degrees. Ascendants, parallax, rising and setting, and the armillary sphere all depend on this convention. An internally consistent function can still return a wrong result when the caller supplies the wrong clock.

The text's longitudes are sidereal. The implementation uses its own ayaṇāṁśa to obtain a tropical value when a comparison requires one. Location-dependent functions accept latitude and longitude, but the reference meridian in the historical model is not Greenwich. Each numerical example should state its date, local clock, and longitude convention.

The time convention affects tests as well as user input. A modern interface may receive a civil clock time with a UTC offset, whereas the chapter functions reckon from the historical meridian. The chatbot tools convert at this boundary and label the resulting zone. This keeps each chapter function from having to infer whether a fractional day means local civil time, Ujjain mean time, or Universal Time. The DE440 evaluation has a separate reference clock, Terrestrial Time, and therefore computes a conversion involving the Ujjain longitude and ΔT. Reporting these clocks explicitly prevents a time shift from being mistaken for a flaw in planetary theory.

The historical longitude is measured from a sidereal reference. A modern DE440 position begins as a vector in a different frame and becomes a longitude only after a stated transformation. The project's evaluation rotates geometric geocentric vectors to the mean ecliptic and equinox of date, then compares circular angles. The ayaṇāṁśa used for the historical longitude is itself a model with a back-and-forth trepidation. A residual consequently contains both the planetary-model difference and the chosen precession model. Reporting the raw sidereal value separately from the converted tropical value makes this distinction visible to users of the Python API and web tools.

4.2 Calculation architecture

The Python package has modules ch01_mean_motions.py through ch14_reckoning_time.py. A module corresponds to a chapter and exposes its quantities to other modules. Later computations call earlier positional and calendrical functions instead of copying formulas. The dependencies can be followed directly: a mean longitude from Chapter 1 enters a true longitude from Chapter 2, which can then enter an eclipse, conjunction, visibility calculation, or panchanga element.

The planets package regroups chapter functions under Sun, Moon, Mars, Mercury, Jupiter, Venus, Saturn, Rahu and Ketu. It is a convenience interface, not a second planetary model. Ketu is placed opposite Rahu as an interface convention because the text does not compute it independently. The package metadata requires Python 3.9 or later and NumPy as the core runtime dependency. Plotting, web services and model access add separate dependencies at the application layer.

The FastAPI application includes calculation routers and book routers for individual chapters. This closely follows the package structure, so a user-visible result can be traced to a chapter function. The chatbot tool registry also calls those functions and supplies schemas for the model. The MCP server generates its tool list from that same registry. This shared path limits duplicated astronomical logic, although interface formatting and input conversion still require their own tests. The project's architecture document and api/main.py show the current composition; older README diagrams omit some later modules and should not be used as the sole architecture source.

The chapter modules return structured results where a calculation has several intermediate parts. A planetary-position result, for example, can contain mean longitude, true longitude, *manda* and *śīghra* contributions, *ayaṇāṁśa*, sign and *nakṣatra*. The chatbot tool serializes selected fields to JSON, adding human-readable context while preserving the numeric values. The API schemas and React client provide a typed boundary for transporting these results. This helps verification: a failing total can be traced to an intermediate term without reimplementing the whole chapter to find the difference.

4.3 Chapter-wise algorithm implementation

4.3.1 Chapter 1 mean motions

Chapter 1 establishes measures of time, the elapsed day count (*ahargaṇa*) and mean revolutions over a *mahāyuga*. The implementation converts dates to Julian Day, computes the elapsed count and obtains a body's mean longitude from its revolution constant and elapsed fraction of the cycle. It also supplies

calendar quantities reused by the panchanga and later chapters. Even a small difference in these rates accumulates across centuries.

The inner planets require a careful reading of the revolution table. Mercury's and Venus's listed revolutions describe their *śīghroccas*, while the mean planetary place is linked to the mean Sun. Treating those entries as ordinary mean planetary motion makes the later *śīghra* anomaly collapse. The code now represents the distinction explicitly, and a test based on the planets' limited elongation from the Sun guards it. Here, a domain property catches an error that syntax and type checks cannot.

Mean longitude is a cyclical quantity, so the elapsed count and revolution count have to be interpreted together and reduced to a circle. The code has separate constants for the seven classical bodies, the lunar node and apogee. The node moves retrograde and therefore needs a sign convention distinct from ordinary prograde motion. The project's evaluation paper tabulates the adopted revolution counts and their implied rates, which makes secular drift measurable. The comparisons with modern rates evaluate the chosen constants; they do not supply inputs to the historical calculation. Chapter 1 tests also cover calendar and unit quantities that later chapters reuse.

The epoch itself is a mathematical input, not merely a date printed in a preface. Chapter 1's elapsed-day count determines the phase of every mean motion. A one-day mistake in the count has a different angular effect for the Moon, Sun and slower planets. The code therefore includes date conversion functions and tests for epoch and calendar behavior. The project's later local-time correction showed that a correct day count can still be misused when a modern clock function interprets it on another meridian. Tests of mean motion and place-dependent calculations therefore need to check this boundary together.

4.3.2 Chapter 2 true positions

Chapter 2 supplies the 24-entry *jyā* table at radius 3438 and the corrections that move a body from mean to true position. The implementation interpolates tabulated sines and applies the *manda* and *śīghra* corrections in the prescribed half and full sequence. The *manda* anomaly is measured with the sign convention of the text. Reversing it changes the equation of centre's sign and can make a computed solar position several degrees wrong while the calculation still runs normally.

The routines use modern inverse trigonometric and square-root operations, so the project implements the text's model with explicit numerical conventions. It does not reproduce every rounding step of

historical hand arithmetic. The longitude path is particularly important because the same positions feed later eclipse, conjunction and calendar calculations, and because it is the part evaluated against DE440.

For the five star-planets, the *śīghra* correction uses a second moving reference. The Sun supplies that reference for Mars, Jupiter and Saturn, while Mercury and Venus have their own *śīghrocca* motion. The project's true_longitude_full path applies the corrections in sequence instead of adding two independent fixed offsets. The sequence matters because a later correction uses a position already modified by an earlier step. The table of *jyā* values supplies the historical trigonometric base, but interpolation and inverse operations are numerical choices in the implementation. A comparison with a modern longitude therefore measures the complete adopted path.

The code also computes the Moon and node from their own mean motions. Rahu is an ascending lunar node and is represented by retrograde mean longitude; the text supplies no corresponding periodic equation. The API therefore should not describe Rahu's value as an observed point in the same sense as a planet's true place. Ketu is derived at the interface as the opposite point. These distinctions matter when evaluating the node against a modern osculating reference: a difference can reflect the historical mean-node model and the reference definition, as well as numerical implementation.

4.3.3 Chapter 3 direction place and time

Chapter 3 turns a celestial position into local quantities for an observer. The implemented functions cover direction, gnomon shadows, rising and setting and the *lagna*. These calculations combine position with latitude, longitude and the local clock. Tests of these calculations must therefore check more than a body's longitude. The same value of JD interpreted on different meridians changes sidereal time and hence the relation between the ecliptic and the horizon.

Correcting the Ujjain time base was one of the project's most consequential fixes. Before it, code for later parallax and instrument calculations could appear self-consistent while using a sky rotated relative to the observer. A horizon, equator and ecliptic diagram clarifies why a planetary longitude alone cannot determine a local rise time or ascendant.

The gnomon calculation connects a measured shadow with solar altitude. The *lagna* locates the ecliptic point at the horizon, and rising or setting computations locate the horizon crossing of an object. These are related through the observer's local frame but are not interchangeable outputs. The tests for this module use known geometric relations and chapter examples. The current project guide contains a

stale Chapter 3 sample calling functions that do not exist; ch03_direction_place_time.py is the source for the current public functions. A computational edition needs to distinguish documentation from verified code.

The chapter's local calculations connect an abstract celestial coordinate to an observable direction. A longitude identifies a point on the ecliptic, but a horizon crossing also depends on obliquity, local sidereal time and latitude. The project uses the text's obliquity convention in its historical computations. The implementation manuscript reports that mixing the two clocks changed local geometry so strongly that a solar-eclipse parallax example exposed the error. This example shows why tests of astronomical software should include the reference frame and time base, not merely a date string and an expected angle.

4.3.4 Chapters 4 and 5 eclipses

Chapter 4 derives lunar eclipse circumstances from the Moon, Sun and shadow. Chapter 5 treats solar eclipses, which need observer-dependent corrections because the Moon's apparent position changes with location. The implementation includes solar parallax terms *lambana* and *nati*. It distinguishes a geocentric computation from one that uses the observer's place. A result should be reported with the chosen place and date convention, because an eclipse visible at one location need not have the same circumstances at another.

The implementation has known limits. Full three-pass *asakṛt* iteration from a worked commentary example is absent, and some disc-diameter helpers retain fixed mean values. These details can affect contact estimates. The chapter implementation is therefore a useful executable reading of the adopted procedures, while any claim about event-timing accuracy would require a separate, dedicated evaluation.

Eclipse calculations bring several scales together: the difference between solar and lunar longitude, the node's place, apparent diameters and, for a solar eclipse, local parallax. A small mistake in one quantity can shift a contact or change whether a local eclipse is predicted. Chapter 4 and Chapter 5 keep lunar and solar eclipse paths separate because Earth's shadow and the observer's topocentric Moon are different geometries. The worked-example suite checks intermediate quantities such as lambana and nati against printed values. The limitations above restrict what agreement at one example date can establish.

The lunar and solar cases also differ in how parallax enters. For a lunar eclipse, the Earth's shadow and Moon are central to the geometry. For a solar eclipse, the apparent Moon depends on the observer's position on Earth, so a local result needs latitude and longitude. The project's route and interface layers expose both kinds of result. Its own evaluation paper confines its DE440 statistics to planetary longitudes and explicitly excludes eclipse timing. A degree-scale longitude comparison over ten-day samples cannot validate minute-scale ingress and egress predictions, so this boundary needs to remain clear.

4.3.5 Chapter 6 eclipse projection

Chapter 6 converts eclipse quantities into a projected geometry with contact points and diagram data. Separating the projection from the event calculation helps with both testing and explanation. A numerical calculation can be checked before it is rendered, while a figure can make the geometry easier to understand. The project provides plotting support as an optional part of the software; users of the core numerical library do not need it for planetary longitudes.

To interpret an eclipse figure, a reader needs its lines and discs identified, its date and observer specified, and a statement of whether it shows a historical-model prediction or a modern reference. A diagram records the geometry the program drew and must be read with the adopted eclipse assumptions. The chapter is a distinct computational layer that turns earlier angular quantities into an observable configuration.

The projection module is particularly useful for explaining an eclipse result to a reader who does not know the traditional terms. Contact points, discs and relative paths provide a geometric check on signs and directions: a numerical magnitude alone may hide a reversed axis or misplaced shadow. The module can return diagram data separately from plotting, so the core calculation can be tested without a graphics backend. A projection figure can show the chapter result with a caption naming the calculation and its assumptions.

The projection output can also help validate the calculation. The relative path of the eclipsing discs should agree in direction with the signs of the computed latitude and contact offsets. If a plotted path moves the wrong way, the defect may be in coordinate conversion or plotting rather than the event search. Separating numerical geometry from optional Matplotlib drawing lets tests focus on the underlying values. In an academic presentation, a projection figure can accompany a small table of the same contact quantities, letting the reader compare the diagram with numbers.

4.3.6 Chapters 7 and 8 conjunctions and stars

Chapter 7 searches for conjunctions between planets and between a planet and a junction star. Chapter 8 contains star and *nakṣatra* data, including functions that connect a longitude to an asterism and calculate star rise or set. A conjunction is an event defined through changing relative position, so the implementation must search over time rather than calculate only a single date's value. The catalogue layer provides named objects and their coordinates for these searches.

The coordinates of the junction stars are an interpretive issue. Chapter 8 uses an ecliptic reading of the catalogue, while Chapter 13 uses a polar-coordinate rule to place stellar hoops on the armillary sphere. The two readings give different declinations for some stars. The project preserves the distinction instead of forcing every use of the catalogue through one unexamined coordinate interpretation.

Chapter 8's catalogue also links computational values to textual names. The *nakṣatra* lookup divides the circle into asterisms and portions, while the fixed-star functions use catalogue coordinates. The computational-edition study derives the listed *dhruvaka* values from the shares given in the verses and records the small effect of catalogue rounding. In Chapter 13, a different construction is needed for stellar hoops. Running these readings side by side allows a comparison for a particular star and date without implying that the historical coordinate question has been settled.

The nakshatra_catalog.json data and star functions give the catalogue a defined software representation. A verse-based star entry can be inspected through the corpus browser while its coordinates enter a calculation. The computational-edition manuscript notes that the *dhruvaka* values for twenty-seven stars follow a rule based on each star's share and the accumulated minutes of preceding asterisms. An apparent small discrepancy for Revatī is explained by rounding in the stored catalogue. This example shows how a tabulated historical value, a derivation and a machine representation can agree only to a stated precision.

4.3.7 Chapters 9 and 10 visibility and the Moon

Chapter 9 treats heliacal rising and setting: the return of a planet or star to visibility after it has been close to the Sun. This is not the same event as a geometric conjunction. The library therefore provides dedicated functions for finding the next heliacal event. Chapter 10 addresses the Moon's elongation, illuminated fraction, phase name, crescent width, horn angle, moonrise and moonset. The calculations combine angular position with local geometry where necessary.

These chapters show how a shared positional core supports questions that require additional assumptions. A phase can be inferred from Sun–Moon elongation, while a rise or set time needs a place. A claim of naked-eye visibility is more demanding than a geometric calculation and should be phrased according to the particular rule implemented. The chapter modules make these distinctions clear to callers.

The Moon functions produce several kinds of output from the same Sun–Moon geometry. Elongation is an angle, illuminated fraction is a geometric consequence of phase, and a crescent's width and tilt depend on further relations. Moonrise and moonset also require the observer. The project's guide gives an end-to-end example for these Chapter 10 calls, while tests exercise the implementation. For heliacal events, the code searches forward from a starting JD rather than requiring the user to guess the event date. This design is suitable for an exploratory interface, provided that the output is explained as the result of the adopted visibility criterion.

The applications make these outputs available in several forms. A positions page can show the Sun and Moon on a zodiac wheel; a Moon page can show phase and crescent geometry; a star page can use a sky map. These visuals explain the same chapter results. They also expose where an input is missing. A phase calculation can run for a date alone, while moonrise needs a site. The chatbot's context includes a user place and UTC offset when available, and its tools are instructed to state what location and clock they used.

4.3.8 Chapter 11 patas

Chapter 11 implements Vyatipāta and Vaidhṛta as *pāta* events associated with equal declinations. The calculation corrects lunar declination for latitude, searches for the event and estimates its beginning and end from the relevant motion. This chapter shows why a name alone should not define an algorithm. The *pāta* here differs from the daily panchanga *nitya-yoga* terms that share the names Vyatipāta and Vaidhṛta.

A verse-level check changed an earlier duration computation: the denominator is the difference of daily motions required by the passage, not simply the rate at which two already-computed declinations separate. This difference has a material effect on the duration. The current module, API routes and user-interface page make the event inspectable while tests preserve the adopted interpretation.

The Chapter 11 module is exposed through find_patas and compute_pata_details chatbot tools, calculation routes and a Patas page. These interfaces allow checks that all outputs share the same meaning. The event has a true instant and an interval; the interval depends on the rate specified in the verse, so a mistaken denominator changes duration even if the central instant looks plausible. The chapter's separate name and tests also help prevent confusion with panchanga *yoga*, where a similarly named daily category is computed from the sum of solar and lunar longitudes.

The *pāta* calculation shows how a derived event can magnify a small definitional mistake. The centre of the event is found by a condition on declinations, but the start and end depend on how quickly the relevant quantities change. A rate based on the wrong motion makes an interval too short even if the central condition is still reached. The implementation changelog records the verse-based correction and the corresponding API and UI exposure. Its tests can separately check the declination relation at the true instant and the interval arithmetic. Checking these separately makes the numerical result easier to audit.

4.3.9 Chapter 12 cosmography

Chapter 12 treats the Earth's globe, distances in the text's cosmic model and cycles of time. The software derives planetary orbit distances from the Moon's stated 324,000 yojanas and the assumption that each body covers the same linear distance in a yuga. It also implements the Earth's circumference, a latitude-adjusted circumference and a longitude difference correction needed elsewhere in the book. These are computations within the *Sūrya Siddhānta*'s model. They should not be described as modern measured distances.

The module also represents descriptive material through a verse-tagged catalogue. Separating numerical derivations from descriptive entries lets an interface present the source without implying that every cosmographic statement is a contemporary astronomical measurement. The chapter shows how the project handles material that is partly procedural and partly explanatory.

The cosmography calculation reproduces stated orbit values from a common linear-distance assumption and the Chapter 1 revolution counts. The stated Moon orbit, shared rule and derived orbit values can be compared directly with units. The project changelog notes that the derived values reproduce the text's figures closely and connect the Sun orbit to a constant already used in the lunar-eclipse module. That connection is an internal consistency check on the edition. It does not turn the yojana values into empirical modern distances.

Chapter 12 also includes material that a web page can display without turning it into a numeric calculation: creation order, cosmic time divisions and other descriptive entries. The data catalogue tags these entries by verse. The numerical functions cover Earth dimensions, orbit derivation and time regents. In the project changelog, the derived orbit values agree with the stated values to within about 1.3 yojanas, a precise internal check for the implemented rule. The figure is a result of arithmetic and rounding in the historical model; it should not be confused with uncertainty in modern physical distances.

4.3.10 Chapter 13 instruments

Chapter 13 describes instruments rather than only a sequence of planetary formulas. The implementation represents the *gola-yantra* as a parametric armillary sphere in an equatorial frame. Hoops for the equator, ecliptic, horizon, colures, solar paths and stars can be passed to a renderer. The React interface contains an interactive three-dimensional representation, while the underlying module stores the astronomical geometry independently of the graphics library.

The chapter also implements the clepsydra and time finding from the gnomon, wheel, arc and staff. Some of these procedures are supplied by Burgess's commentary rather than fully by the verses. The output therefore identifies its source category. This allows a user to distinguish an operation directly warranted by the text from a useful reconstruction that completes an underspecified description.

The armillary model stores a hoop by its plane and offset, so a renderer can construct a sphere without recomputing astronomy. It changes with place and moment: horizon orientation and the ecliptic point on it are local, while some reference circles remain fixed in the model frame. The interactive page makes this movement visible. The instruments section also shows why source labelling matters. Where a verse names a device but does not give every step needed to calculate time from it, the software may use Burgess's commentary, but the result says so. Stating the added rule presents that uncertainty more faithfully than hiding it.

The Chapter 13 model uses an equatorial coordinate frame whose north pole and equinox axes are fixed by convention. A hoop is a geometric object in that frame, and a page renderer can rotate the view without changing the underlying astronomical values. For Ujjain, the computational-edition manuscript reports 46 hoops in its model. It also states that the eastern-horizon ecliptic point is computed directly instead of interpolated between zodiac signs. This matters because interpolation

could make the interactive sphere look smooth while giving a measurably wrong local ascendant. The implementation separates the geometry from its display.

4.3.11 Chapter 14 reckoning of time

Chapter 14 returns to calendrical questions. The implementation covers measures of time, solar ingress, months and seasons, and named Jovian years. These operations draw on earlier solar positions and the historical day count, so they inherit the date and meridian conventions already discussed. Their outputs often use familiar calendar labels, but each label comes from a calculation with a particular rule and epoch.

The chapter also includes alternative ways of naming a Jovian year. The verse rule and the mean-longitude method supplied in commentary remain distinct where both are exposed. This follows the project's approach of preserving competing procedures and their provenance when a single answer would hide an interpretive decision.

The code supplies time-unit conversion through a catalogue of *mānas* and calculates solar ingress events for a Gregorian year. It can also return ayanas, equinox-related crossings and *ṣaḍaśītimukha* divisions. The Fathers' day calculation and Jupiter's named-year cycle have separate methods. The barhaspatya_year function lets a caller select a method; the result therefore needs to identify the method as well as the year name. Chapter 14 handles calendrical categories whose definitions depend on the textual source, rather than treating a familiar modern calendar label as self-explanatory.

The time-reckoning chapter's solar_year function is organized around the Sun's entry into Aries in a Gregorian year. Related functions return the ingress events and seasonal divisions, while convert handles numerical conversion between measures of time. The *pitrya_clock* function addresses the Fathers' day using the Sun–Moon relation. These are distinct routes through the chapter; they should not be summarized as a single generic "calendar converter." The named Jovian year also needs a method label, because a mean-motion method in Burgess's discussion and a method tied to heliacal behavior need not return the same label at every date.

4.4 Public interfaces

The project developed several interfaces around the same library. Earlier Gradio and Streamlit applications present planetary positions, panchanga, eclipses and other calculation groups. The current React application uses FastAPI for calculations, chapter browsing, corpus search, glossary access and

chat. These interfaces have different feature sets, but their astronomical results come from the chapter functions.

Using one calculation layer keeps the interfaces consistent. Input conversion, formatting and response handling still need their own checks. A screenshot shows what a user can inspect for specified inputs; it does not establish that the underlying astronomy is correct. Source comparisons, worked examples and tests provide that evidence.

The React application includes later-chapter pages such as Cosmography and Instruments, together with chapter and verse browsing, learning paths and research views. SVG and three-dimensional components illustrate the geometry. The older Streamlit and Gradio applications remain separate interfaces with their own page sets. The REST layer returns structured results so other clients can use the same calculations.

A user can read a verse, calculate a result for chosen inputs and inspect the relevant geometry. Glossary entries explain unfamiliar terms, and API responses retain numerical details for another program or a test. The learning paths at /learn reuse the Chapter 1 and 2 APIs. The Research Explorer at /research reads the saved evaluation data. Source code and tests record what each calculation actually does.

4.4.1 Technologies used

The technologies serve distinct parts of the project. The calculation library remains independent of the web and model providers. Table 1 summarises the current implementation and the earlier interfaces used during development.

Table 1 Technologies and their roles

Area	Technologies and role
Calculation library	Python and NumPy implement the numerical methods; Matplotlib supports plots and eclipse diagrams.
Backend and contracts	FastAPI, Pydantic and Uvicorn provide REST endpoints, request validation and the ASGI server.
Current web interface	React JS, TypeScript, Vite, React Router, HTML and CSS provide the browser application.

Area	Technologies and role
Geometry and text	Three.js, React Three Fiber and Drei support 3D views. KaTeX, React Markdown and remark-gfm display equations and formatted text.
Earlier interfaces	Streamlit and Gradio were used for the earlier calculation, panchanga and chatbot interfaces.
AI providers and tools	OpenRouter and DeepSeek APIs provide model access through the OpenAI Python SDK. Tool calling and MCP connect models to project functions.
Corpus and retrieval	SQLite FTS5 and BM25 support indexed search. JSON and JSONL hold corpus exports and processing records.
OCR and translation	Gemma 4 E2B, Surya OCR 2, Sarvam Document Intelligence and Patram were explored for document extraction and translation.
Model runtimes	LM Studio and Hugging Face Transformers support local model work; the Transformers path uses PyTorch.
Document preparation	PyMuPDF, OpenCV and Pillow support PDF extraction and page-image processing. python-docx and Markdown outputs support document export.
Checks and delivery	pytest and Playwright support software checks. Docker and Docker Compose package the services; Git and GitHub track and share the source.

4.5 Corpus preparation retrieval and chatbot

4.5.1 Source extraction and corpus construction

The separate ss_corpus project prepared machine-readable material from the English Burgess edition and the scanned Hindi edition. It combined embedded-text extraction, page-image OCR, cleaning and verse segmentation. Hindi preparation also checked legacy fonts and converted suitable embedded text to Unicode. The output could then be inspected by page before it entered a verse record.

Several model paths were explored because dense prose, Devanagari characters, equations and tables create different extraction problems. Table 2 describes the runs and outputs found in the project. These experiments do not establish a controlled accuracy ranking between models.

Table 2 OCR and translation work

Model or service	Recorded work
Gemma 4 E2B	The saved gemma4_v2 pass records Hindi OCR and page-level English translation for all 838 scanned pages.

Model or service	Recorded work
Surya OCR 2	The saved run records OCR for all 838 pages, with page Markdown and Word exports.
Sarvam Document Intelligence	Notebook runs produced combined Hindi HTML, Markdown, text and a reflowed PDF.
Patram	Notebook experiments explored transcription and translation of scanned Indic document pages.

The gemma4_v2 progress record shows 838 of 838 pages completed for both OCR and page-level English translation. The Surya OCR record also shows 838 completed pages. Page caches, progress logs and combined outputs allow a run to resume without repeating finished pages. Completion means that the pages were processed; it does not mean that every word, formula or translation has been independently verified. A separate paragraph-based Word translation run still has pending units and should not be confused with the completed page-level pass.

For the English verse corpus, the pipeline cleans OCR, identifies chapter and verse boundaries, separates translated verse text from commentary, adds topics and maps relevant passages to calculation tools. Records include a stable verse ID, chapter and verse numbers, language fields and source metadata. JSON and JSONL exports preserve the records; SQLite FTS5 provides indexed search with BM25 ranking.

The application uses a corrected Burgess export with 500 numbered English verses across all fourteen chapters. Source-page checks restored missing verses and repaired verse/commentary boundaries. The earlier final export in ss_corpus currently contains 455 records, so it must be distinguished from the repaired export bundled with this application. The bundled records have no populated Hindi translation fields. Hindi OCR and English translations of the Hindi book are separate outputs, rather than a completed verse-aligned bilingual corpus.

The corpus supports chapter browsing, verse search and source retrieval. It does not supply modern planetary positions to the historical calculations. The chapter functions remain responsible for astronomical results. Complete numbered-verse coverage also does not imply error-free transcription or a complete critical edition.

4.5.2 Retrieval and AI query interface

The chatbot combines calculation tools with retrieval of chapter- and verse-indexed text. For a numerical question, its instructions require a calculation tool call. Before quoting or explaining the book, it should retrieve the relevant passage and retain its verse ID and source metadata. The retrieval layer can use an indexed SQLite database or build an FTS5 index from the bundled JSON export.

Chat retrieval, corpus search and Verse Explorer use the corrected 500-verse Burgess export by default. Explicit JSON and database overrides remain available. Source locators and repaired verse boundaries help readers check a citation, although unreviewed OCR can still contain errors. Optional Vigyan Bhashya commentary remains a separate source. The assistant uses OpenRouter, direct DeepSeek or a custom compatible endpoint through the OpenAI Python SDK. The SDK is a client library and does not imply use of an OpenAI-hosted model. MCP exposes the same calculation and retrieval tool registry over standard input and output; the connected client supplies its model.

Retrieval supports exact verse lookup, chapter ranges, search and a chapter index. Returned verse IDs give the assistant a consistent way to cite a passage. Burgess's translated verses and Srivastava's commentary must be labelled separately. Numerical answers should come from the calculation functions rather than model recall or a number copied from OCR. These instructions reduce avoidable errors, but they do not guarantee a correct final explanation. A useful demonstration records the question, tool call, retrieved verse and final response.

The tool loop has a bounded number of steps. When a supplied SQLite index is unavailable or empty, retrieval falls back to an index built from the bundled export. The app can therefore search and cite the Burgess corpus without running the external OCR pipeline. The quality of an explanation still depends on the retrieved passage, commentary coverage and transcription accuracy.

4.6 Test strategy

Verification has three layers. Verse-anchored tests ask whether each function follows the adopted rule. Worked-example checks ask whether a chain of calculations reproduces an example at its printed precision. The DE440 comparison asks how far a particular longitude path departs from a modern reference under specified conventions. Each layer catches a different kind of problem. A code test can pass even when the chosen reading of a verse is wrong; a modern comparison can be close by chance; a printed example may contain a typographical or OCR error.

The computational-edition manuscript reports 1,256 chapter and per-planet interface tests and 257 selected worked-example cases. It also states that the audit-case counts refer to an audit branch whose cases and exact revision have not yet been archived. These are reported project results, not a newly executed count from this checkout. A future release can record a fixed code revision and the exact test command alongside the audit cases.

The strongest tests combine numerical assertions with source-based properties. An elongation bound catches a semantically wrong inner-planet formula. A printed eclipse example tests intermediate parallax, which can catch a clock shift. Wrapper tests check that the per-planet entry points remain aligned with their chapter functions. The repository also includes API and chatbot tests, so the interface path can be checked independently of the chapter mathematics. A test count belongs to a particular date and revision because this repository is still being developed. The description of how each class of test relates to its sources remains valid even when the count changes.

The test organization follows the chapter structure, with dedicated files for Chapters 1–14 and additional files for API, chatbot, corpus and per-planet interfaces. The computational-edition manuscript separates chapter tests from wrapper tests, and a recent DE440 revalidation ran a focused Chapter 1–2 suite of 263 tests. Those figures refer to particular checks and revisions, so they should not be added together as a new total. A unit test should encode a textual or geometric expectation, while a worked-example test should preserve the printed inputs and output precision.

4.7 Learning and research interfaces

4.7.1 Mars Learning Lab

The /learn/mars route turns the Chapter 2 true-place calculation into a guided experiment for Mars. A whole-day input and a slider spanning 90 days on either side of the chosen date control the existing book API. Results are evaluated at midnight in Ujjain mean time and displayed as sidereal angular directions from Earth. The drawing shows angles, not physical orbital distances.

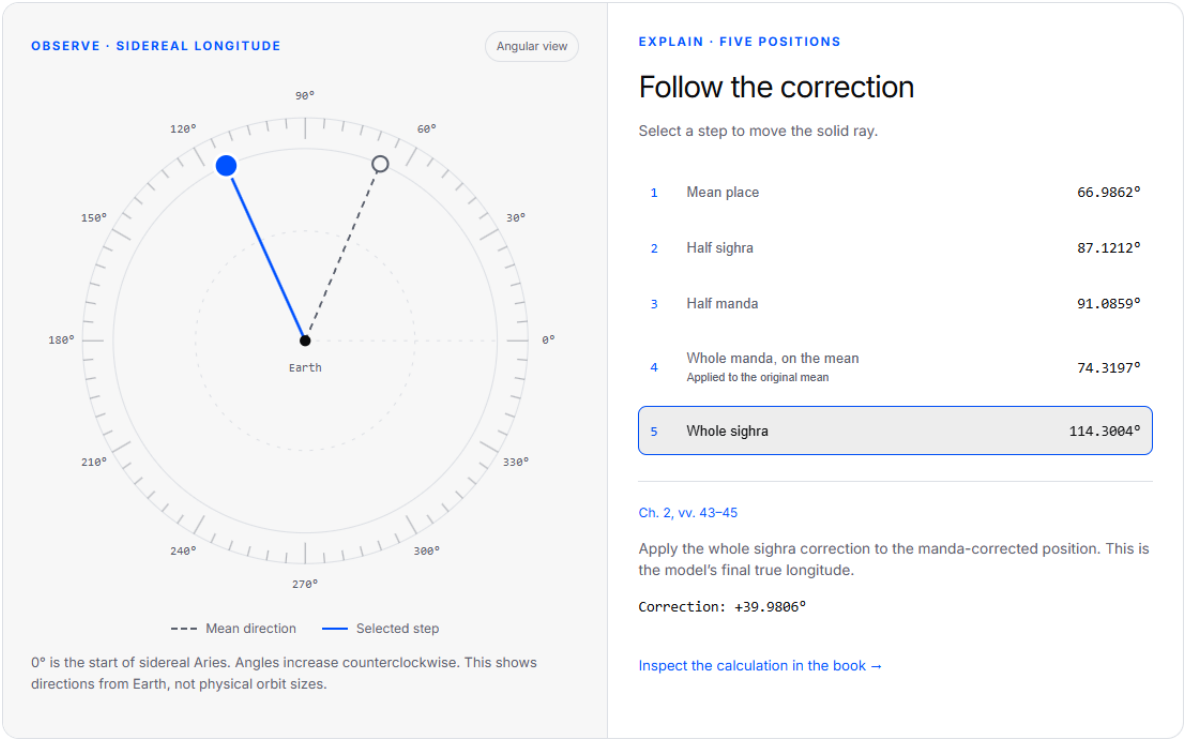


Figure 1 Mars Learning Lab showing the final correction stage for 4 November 2026

The five selectable positions are the mean place, half sighra, half manda, whole manda on the original mean, and whole sighra. The fourth step explicitly returns to the original mean, so the sequence should not be read as a cumulative sum. At the demonstration date the final longitude is 114.3004 degrees. A link opens the source calculation with the same date; optional corpus text is distinguished from calculation availability.

An understanding check asks whether the final position is ahead of or behind the mean along the shortest angular separation. JSON export preserves the inputs, time basis, frame and intermediate results. This teaching functionality is implemented; its effect on student learning remains unmeasured.

4.7.2 Learning paths

The /learn directory contains eight working lessons: Sun, Moon, Mars, Mercury, Venus, Jupiter, Saturn and Rahu. Learners can start with the Sun, then study the Moon before tracing the planetary corrections. The directory supports direct links to any lesson; it does not require a prescribed order or claim measured mastery.

Learning path	Implemented experiment
Sun	Mean and manda-corrected longitude; tropical Sun, declination and day/night length at Ujjain.
Moon	Two correction positions; Sun–Moon elongation, 12-degree tithi intervals and waxing/waning paksha.
Mars	The original five-position lesson and its saved research panel.
Mercury and Venus	Individual lessons and a same-date comparison of every correction stage and Sun separation.
Jupiter and Saturn	Individual lessons and a same-date comparison of the five-position sequence.
Rahu	Ascending and descending lunar nodes, regression, longitude geometry and an eclipse concept quiz.

All seven body lessons share date validation, a direction diagram, selectable API-derived stages, chapter references, a prediction check and JSON export. Dedicated panels relate the calculation to each topic. Sun and Moon have two positions rather than the five used for the star-planets. Planet comparisons use one API response so both bodies share the input date.

The seasonal panel explicitly changes from sidereal to tropical longitude and states that it uses the library's hour-angle day-length method. The Moon panel displays elongation and tithi without claiming to render the illuminated disc. Each tithi spans 12 degrees of longitude separation and need not last one civil day. Paired-planet Sun separation is a longitude difference, not an observational visibility estimate.

Downloads include the lesson's additional context: day-length values for the Sun, tithi results for the Moon, or comparison rows for paired planets. Late responses cannot overwrite a newer date. Invalid or failed requests keep exports disabled and label any previous result. The numerical lessons remain available if the optional corpus fails.

4.7.3 Lunar nodes and eclipse geometry

The Rahu lesson at /learn/rahu draws the ascending lunar node returned by Chapter 1 and derives the opposite descending direction by adding 180 degrees modulo 360. The Sun and Moon positions come from Chapter 2 for the same input date. Learners can select either node or the Moon direction and observe regression through the node's negative mean motion.

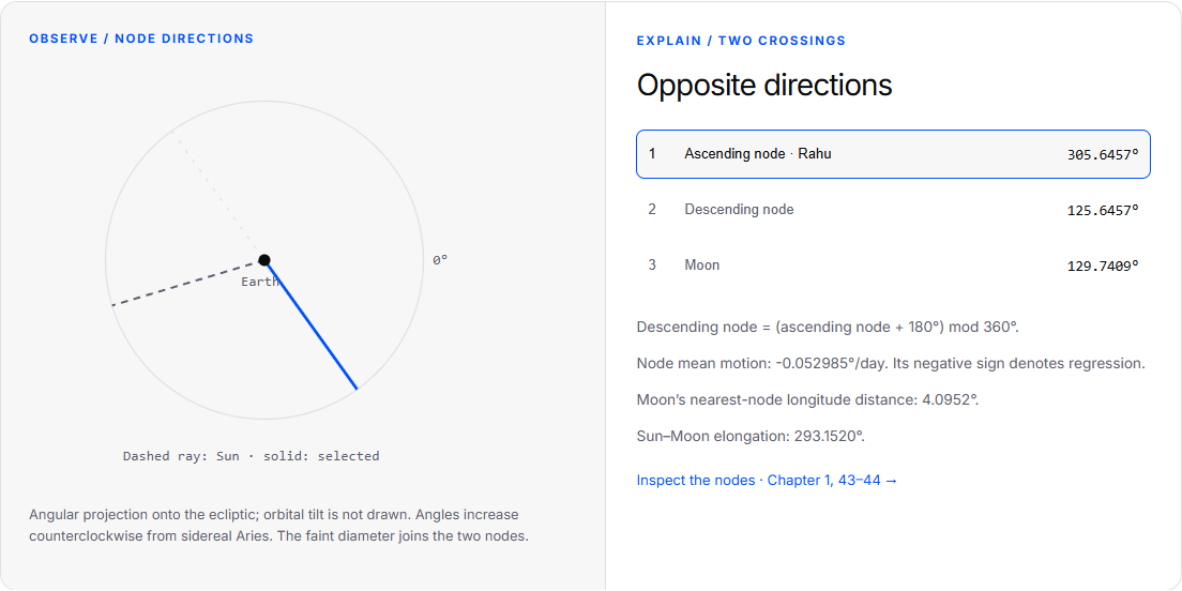


Figure 2 Rahu lesson showing the two lunar nodes and same-date Sun and Moon directions

The projection shows ecliptic longitude only, with orbital tilt described in text. The lesson explains that solar eclipses require new Moon near a node and lunar eclipses require full Moon near a node. Its concept check asks why phase alone is insufficient. The export records event_prediction as false. Determining event occurrence, contacts and local visibility requires further geometry; links lead to the existing eclipse calculators without treating this diagram as an event prediction.

The conceptual background is supported by NASA Science, Eclipses and the Moon (https://science.nasa.gov/moon/eclipses/) and Moon Phases (https://science.nasa.gov/moon/moon-phases/), consulted 4 November 2026. Historical node and tithi references remain the project's Chapter 1 verses 43–44 and Chapter 2 verse 66. Modern explanatory references do not replace the adopted historical model.

4.7.4 Research Explorer

The /research route exposes the saved longitude study for the Sun, Moon, Mercury, Venus, Mars, Jupiter, Saturn and Rahu across eleven 100-year windows from 1550 to 2649 CE. An 8 by 11 RMS matrix allows direct selection; native body and window selectors provide the same choice. Rahu is

identified as the ascending lunar node. The two time treatments share a vertical scale within each selected comparison.



Body	1550 −1649	1650 −1749	1750 −1849	1850 −1949	1950 −2049	2050 −2149	2150 −2249	2250 −2349	2350 −2449	2450 −2549	2550 −2649
Sun	1.14	1.27	1.39	1.53	1.66	1.79	1.93	2.51	5.30	8.41	11.54
Moon	1.69	1.76	1.85	1.95	2.05	2.16	2.29	2.79	5.39	8.46	11.58
Mercury	3.31	3.25	3.20	3.16	3.14	3.13	3.14	3.45	5.78	8.71	11.75
Venus	4.16	4.74	5.38	5.96	6.61	7.23	7.77	8.55	10.17	12.58	15.14
Mars	1.88	2.21	2.67	2.96	3.36	3.85	4.16	4.95	7.98	11.20	14.56
Jupiter	2.09	2.24	2.45	2.87	3.33	3.84	4.29	4.19	2.12	1.94	4.45
Saturn	4.88	4.90	5.70	6.35	7.44	8.77	9.45	10.67	13.56	16.65	20.04
Rahu	1.50	1.79	2.10	2.42	2.76	3.10	3.43	3.43	1.63	2.16	4.56

Figure 3 Time-aligned RMS longitude errors from the saved DE440 comparison

The naive treatment reads library JD directly as reference TT. The time-aligned treatment subtracts 75.7873/360 day for the meridian and adds the paper's approximate Delta-T in days. It changes the reference epoch, not the historical formulas. For the Moon in 1550 to 1649, the saved RMS changes from 3.990 to 1.686 degrees. A lower RMS in one window is a diagnostic result, not calibration of the SS model.

The chart preserves the minimum and maximum sampled residual in each five-year group and connects the group means. Full-window statistics use every original ten-day sample. JSON and CSV downloads include source paths and SHA-256 hashes, frame and time conventions, and sample counts. The explorer does not need a live JPL request.

4.7.5 Universe Explorer

The Universe Explorer provides a 2D map and a 3D view of the available celestial bodies. Users can change the date, inspect a selected body and follow guided tours. Three.js, React Three Fiber and Drei provide the 3D scene; a 2D view remains available where WebGL cannot be used.

The interface distinguishes the historical model from the modern comparison when the latter is available. A timeline and trajectory views support exploration of motion, including a Mars retrograde

lesson. Body details and source links connect the visual view to the existing learning and book pages. The scene is an explanatory representation; its appearance alone does not validate a calculation or establish historical orbital distances.

5. Results And Findings

5.1 Worked-example case studies

The inner-planet correction shows how a program can run with the wrong interpretation. Treating Mercury's and Venus's revolution counts as their own mean motion made the *śīghra* anomaly zero and placed Mercury 153 degrees from the Sun on one tested date. A bound on elongation exposed the mistake. Another check fixed the sign of the *manda* anomaly: reversing apsis minus planet changes the correction and can shift the Sun's computed place by several degrees.

A Kashi solar-eclipse example exposed the time-base error. With Ujjain local time and Universal Time mixed, first-pass longitude parallax came out at 3.83 *ghaṭikās*, while Srivastava printed 1.06. A consistent clock brought the computed value to 1.16. The remaining difference matters because the commentary uses a more complete iteration and a different disc-size treatment. These cases justify keeping intermediate quantities and source rules visible, instead of judging the software by a final answer alone.

The case studies record both the symptom and its cause. For Mercury and Venus, an impossible elongation came from confusing a sighrocca revolution count with mean planetary motion. The manda correction used the anomaly in the wrong direction. The Kashi example mixed time conventions. Each correction led to a more specific test, so the interpretation can be checked again when the code changes.

Another case involves the lunar apparent diameter. An initially plausible scaling placed the largest disc at apogee; the verse-based calculation scales the mean diameter by the ratio of true to mean daily motion. The worked example's intermediate numbers exposed the reversal. Chapter 12 provides a different kind of check: orbit values derived from the shared-distance rule nearly reproduce the printed values. Chapter 14 tests can protect named seasons and years against off-by-one shifts. Together, these cases show why verification needs more than one metric. It uses domain bounds, arithmetic examples and cross-chapter consistency where each is appropriate.

5.2 DE440 evaluation

A companion study compares Chapter 1–2 longitudes of the Sun, Moon, five visible planets and lunar node with JPL DE440. It samples 40,176 epochs at ten-day intervals from 1550 to 2649 CE and considers both a direct clock comparison and a Ujjain/ΔT-corrected variant. Independent revalidation reconstructed all 642,816 stored residual values against the local DE440 kernel to within about

0.00000055 degrees. This verifies the arithmetic of the stored residuals under the stated frame and time conventions. It does not validate every textual choice, later-chapter event prediction or historical panchanga practice.

The comparison obtains tropical longitude by adding the project's ayanamsha to its sidereal longitude. A scalar precession-corrected residual is a diagnostic on longitude, not a full three-dimensional fixed-frame rotation. The text's trepidation model reaches a maximum near 2299 CE and then reverses, affecting future extrapolation. The repository's revalidation note records concerns from an earlier manuscript snapshot; it must be read in the context of its October 2026 review date and revision. The current repository also supplies regenerate_paper.py and REPRODUCE.md. The new explorer does not rerun that pipeline or independently reassess its fitted coefficients.

The sampled DE440 interval lies inside the local kernel's coverage. The study uses geometric geocentric vectors, converts them to a mean ecliptic and equinox of date, and compares wrapped longitude differences. The Moon's node uses a separate reference derived from its orbital angular momentum, so its residual differs physically from a planet's. Future epochs test how the adopted constants and trepidation model extrapolate; they are not observations already made. The current explorer groups the saved decimal-year samples into the same windows as the saved tables, with every exported group checked against those tables.

The earlier revalidation identified twelve epochs whose former decimal-year labels assigned them to the preceding century. That finding shows why residual arithmetic and subsequent grouping need separate checks. The current explorer tests verify all 176 body-window-treatment groups against the current saved tables, along with source hashes, sample coverage and retained extrema. This establishes faithful presentation of the supplied evidence, not an independent rerun of the kernel or a validation of event timing. Angular elongation errors should not be converted directly into claims about eclipse or tithi timing without checking actual event crossings.

5.3 Interface validation and reproducibility

The repository verification record dated 8 November 2026 reports that PYTHONPATH=src python -m pytest -q passed 1,677 tests after the corpus repair. The learning-path validation record dated 4 November reports 1,653 passing tests, an earlier focused run of 49 tests across test_learning_lab_data.py, test_research_explorer_data.py and test_api_book_ch02.py, and a successful production frontend build. These results come from dated repository records; the tests were not rerun while this report was edited.

Check	Evidence
Source-data integrity	All 176 body-window-treatment groups match the saved tables; each body has 40,176 samples per treatment.
Aggregation and provenance	Source hashes, complete sample counts, RMS, signed means and sampled extrema are checked; generated files must match fresh exports.
Browser workflows	Real API values, correction stages, whole-day stepping, date-preserving links, rapid responses, failure/retry, optional corpus, understanding feedback and exports.
Research interaction	88 matrix cells; all 16 body/treatment combinations; JSON and CSV metadata; keyboard selection and mobile overflow checks.
Visual review	Light and dark desktop layouts and a 390-pixel mobile layout; screenshot evidence retained with the project.

The frontend data exporters used for reproducibility are docs/evaluation/export_learning_lab.py and export_research_explorer.py. They summarize saved CSVs; they do not compute fresh DE440 reference positions. The source-study pipeline remains separately documented in docs/evaluation/REPRODUCE.md.

Browser checks in frontend cover all eight lessons and the research explorer. They verify API values, lesson evidence exports, node geometry, quizzes, date-preserving source links, stale responses, retry and mobile layout. The calculation remains usable when corpus text is missing. These checks establish the tested software behavior; historical authenticity, eclipse timing accuracy and educational effectiveness require separate evidence.

5.4 Implementation size

An inspection on 13 November 2026 counted the registered API operations and table render locations in the current source checkout. These figures describe implementation size, rather than numerical accuracy, test coverage or the number of tables visible on one screen.

Table 3 API and frontend inventory

Measure	Count and definition
Backend API endpoints	56 method-and-path operations: 46 POST and 10 GET. This includes 54 application operations, the service index and the health check. Automatically generated documentation routes are excluded.
Frontend table placements	167 static JSX render locations: 103 KVTable uses, 58 GridTable uses and 6 direct HTML tables. The two reusable table definitions are excluded.
Counting basis	FastAPI app.openapi() for endpoints; the TypeScript compiler AST for frontend/src TSX files. The figures describe the checkout inspected on 13 November 2026, not a benchmark.

A table placement may be conditional or may render more than once when a component is reused or iterated. It is therefore not a count of unique datasets, database tables or simultaneously displayed tables. The earlier figure of 49 belongs to the focused Python test run described in Section 5.3 and must not be presented as an API count.

6. Summary And Future Plan Of Work

6.1 Summary of work completed

The project delivers a reusable calculation library covering the computable core of all fourteen chapters, together with source-text preparation, retrieval and user interfaces. Chapter and planet imports provide access to the same functions. The corpus records let users inspect the adopted rules, while the AI assistant calls the calculation and retrieval tools. Eight learning paths, the Universe Explorer and the Research Explorer support different ways of examining the results. The worked examples exposed several interpretation and time-base errors. The DE440 comparison measures the longitude differences of the adopted implementation under stated conventions. The Hindi OCR and translation work is documented separately from the English verse corpus currently bundled with the app.

6.2 Limitations and next work

The library covers the computable core of all fourteen chapters, not every verse. Remaining gaps listed in the computational-edition manuscript include full eclipse iteration, some disc-diameter and visibility rules, parts of the equation of time and descriptive passages that are not algorithms. The bundled Burgess corpus has complete numbered-verse coverage, while remaining OCR noise and the optional external commentary still require review. The interpretation of star coordinates and procedures completed from commentary are documented choices rather than uniquely settled historical facts. Modern trigonometric operations also mean the software does not simulate every rounding step of historical hand calculation.

The next steps are to archive the worked-example audit at a fixed revision, independently rerun the DE440 pipeline and extend procedures for which a defensible source rule can be established. Hindi text still needs reliable verse alignment and review before it can be treated as an integrated bilingual verse corpus. The eight learning paths cover the Sun, Moon, five classical planets and Rahu. A student study is needed to assess whether they improve learning.

The text is partly computational, partly descriptive. Completing every verse as a function would require inventing a numerical rule for passages that do not provide one. The project therefore makes a narrower, more testable claim: it covers the computable core across all chapters, with listed exceptions. Improvements should proceed through explicit source decisions. For example, a full eclipse iteration can be added with tests against the commentary's intermediate passes; a revised star-coordinate reading can be compared against both current methods; and the available DE440 pipeline can be independently rerun to regenerate each table from a fixed kernel and code revision. The modular structure allows these changes to be reviewed.

Further changes should be checked where they enter the calculation. Eclipse iteration can be compared with Srivastava's intermediate passes. Alternative star-coordinate readings need explicit comparisons. Corpus revisions need source-page checks and fixed versions. The DE440 tables should be regenerated from a recorded code and kernel revision. These steps would make it easier for another researcher to reproduce the results and assess the remaining differences.

References

[1] Ebenezer Burgess, “Translation of the Sūrya-Siddhānta, a Text-Book of Hindu Astronomy; with Notes and an Appendix,” Journal of the American Oriental Society 6 (1860): 141–498.

[2] Mahavir Prasad Srivastava, Sūrya Siddhānta: Vigyan Bhashya, 2 vols., 2nd ed. (Allahabad: Dr. Ratnakumari Svadhyaya Sansthan, 1982–83).

[3] Kim Plofker, Mathematics in India (Princeton University Press, 2009).

[4] Ryan S. Park et al., “The JPL Planetary and Lunar Ephemerides DE440 and DE441,” Astronomical Journal 161, no. 3 (2021): 105, doi:10.3847/1538-3881/abd414.

[5] Pandu Santhoju et al., “Mathematical Model as per the Algorithm of Sūryasiddhānta for Computation of Location Specific True Position of the Planets,” Journal of Astrophysics and Astronomy 46 (2025): 9, doi:10.1007/s12036-024-10018-8.

[6] Ashutosh Nigam and Dr. Dinesh Mohan Joshi, “Evaluating the Accuracy of Surya Siddhanta Planetary Algorithms against JPL DE440, 1550–2649 CE,” unpublished project manuscript (2026).

[7] Ashutosh Nigam and Dr. Dinesh Mohan Joshi, “Running the Text: A Verse-Traceable Computational Edition Across All Fourteen Chapters of the Surya Siddhanta, and What Implementing It Reveals,” unpublished project manuscript (2026). Computational-edition manuscript

[8] Surya Siddhanta calculation library and application source. Project GitHub repository

[9] SS corpus extraction, OCR, translation and indexing pipeline. SS corpus GitHub repository

[10] Burgess corpus coverage audit and source-checked repair record. Corpus coverage audit

Companion manuscript listed in [6]. DE440 evaluation manuscript